

Multimedia and Hypermedia. Introduction to REST and HATEOAS

Trayan Iliev

IPT – Intellectual Products & Technologies
e-mail: tiliev@iproduct.org
web: <http://www.iproduct.org>

Oracle®, Java™ and JavaScript™ are trademarks or registered trademarks of Oracle and/or its affiliates.
Other names may be trademarks of their respective owners.



Licensed under the **Creative Commons Attribution-NonCommercial-NoDerivs 3.0**
Unported License

Agenda

- Multimedia and Hypermedia – basic concepts
- Service Oriented Architecture (SOA),
- Cloud computing and client side mashups
- REpresentational State Transfer (REST)
- Hypermedia As The Engine Of Application State (HATEOAS)
- New Link HTTP header
- Richardson Maturity Model of Web Applications



Question 1



How many of you know what is
Hypermedia?



Question 2



How many people know what
REST means?



Question 3



How many know what HATEOAS
stands for?



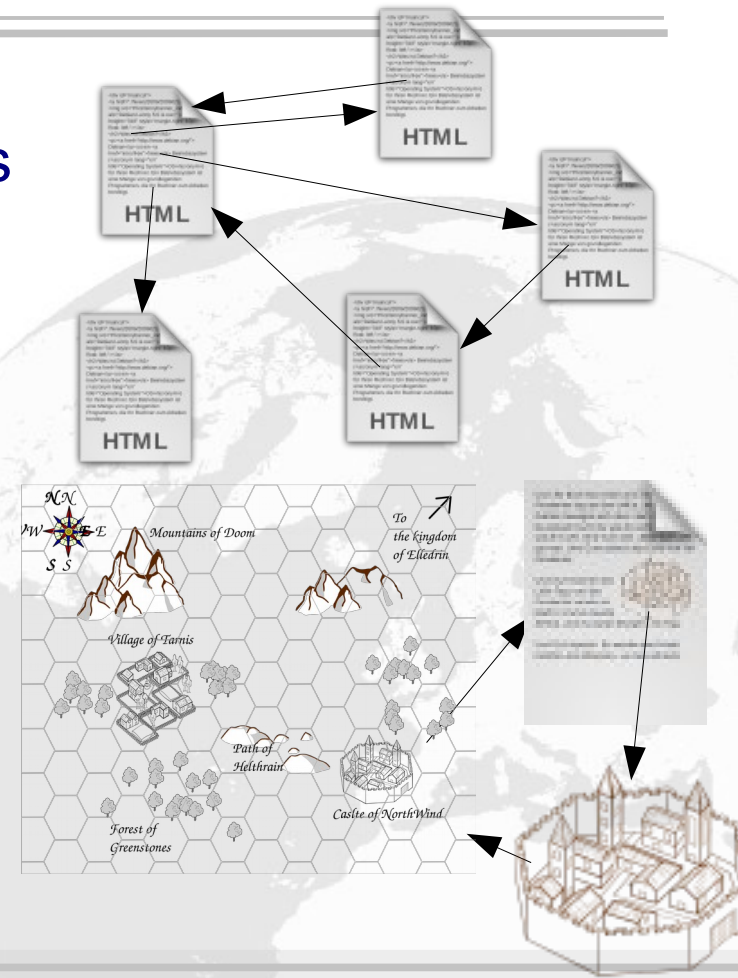
Media Types. Multimedia

- **Text** – a linear sequence of character data
- **Graphics** – raster and vector images
- **Animation** – sequence of changing images (frames) with different frame-rates
- **Audio** – can be discretized analog signal (e.g. MP3) or just musical notes (MIDI)
- **Video** – different file formats, can be linear or interactive
- **3D Graphics / Animation** – using 3D modelling and rendering techniques to achieve realistic, real-world like visualization
- And more – haptic feedback, smells, ...
- **Multimedia** – combining different media types in a coherent and consistent way to achieve better/more realistic user experience



Hypertext & Hypermedia

- **Hypertext** is structured text that uses **logical links (hyperlinks)** between **nodes** containing text
- **HTTP** is the protocol to exchange or transfer hypertext
- **Hypermedia** - extension of the term hypertext, is a nonlinear medium of information which includes multimedia (text, graphics, audio, video, etc.) and hyperlinks of different media types (e.g. image or animation/video fragment can be linked to a detailed description).



Multipurpose Internet Mail Extensions (MIME)

- Different types of media are represented using different text/binary encoding formats – for example:
 - Text -> plain, html, xml ...
 - Image (Graphics) -> gif, png, jpeg, svg ...
 - Audio & Video -> mp3, ogg, webm ...
- Multipurpose Internet Mail Extensions (MIME) allows the client to recognize how to handle/present the particular multimedia asset/node: **Media Type** **Media SubType (format)**
Ex.: **Content-Type: text/plain**
- More examples for standard MIME types:
https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP/MIME_types
- Vendor specific media (MIME) types: **application/vnd.*+json/xml**



Software Architecture – Definitions [1]

Almost everybody feels at peace with nature: listening to the ocean waves against the shore, by a still lake, in a field of grass, on a windblown heath. One day, when we have learned the timeless way again, we shall feel the same about our towns, and we shall feel as much at peace in them, as we do today walking by the ocean, or stretched out in the long grass of a meadow.

— Christopher Alexander, The Timeless Way of Building (1979)



Software Architecture – Definitions [2]

According to **Dr. Roy Thomas Fielding** [Architectural Styles and the Design of Network-based Software Architectures, 2000]:

A **software architecture** is an abstraction of the **run-time** elements of a software system during some phase of its operation. A system may be composed of many levels of abstraction and many phases of operation, each with its own software architecture.

A software architecture is defined by a configuration of architectural elements - **components, connectors, and data** - constrained in their relationships in order to achieve a desired set of architectural properties.



Software Architecture – Definitions [3]

According to **Dr. Roy Fielding** [Architectural Styles and the Design of Network-based Software Architectures, 2000]:

An **architectural style** is a coordinated set of architectural constraints that restricts the roles/features of architectural elements and the allowed relationships among those elements within any architecture that conforms to that style.

The primary distinction between **Network-based architectures** and software architectures in general is that communication between components is restricted to message passing, or the equivalent of message passing if a more efficient mechanism can be selected at runtime based on the location of components.



Architectural Properties

According to **Dr. Roy Fielding** [Architectural Styles and the Design of Network-based Software Architectures, 2000]:

- Performance
 - Scalability
 - Reliability
 - Simplicity
 - Extensibility
 - Dynamic evolvability
 - Customizability
 - Configurability
 - Visibility
- All of them should be present in a desired Web Architecture and REST architectural style tries to preserve them by consistently applying several **architectural constraints**



Service Oriented Architecture (SOA) – Definitions

Thomas Erl: SOA represents an open, agile, extensible, federated, composable architecture comprised of autonomous, QoS-capable, vendor diverse, interoperable, discoverable, and potentially reusable services, implemented as Web services. SOA can establish an abstraction of business logic and technology, resulting in a loose coupling between these domains. SOA is an evolution of past platforms, preserving successful characteristics of traditional architectures, and bringing with it distinct principles that foster service-orientation in support of a service-oriented enterprise. SOA is ideally standardized throughout an enterprise, but achieving this state requires a planned transition and the support of a still evolving technology set

References: Erl, Thomas. Serviceorientation.org – About the Principles, 2005–06



Licensed under the **Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License**

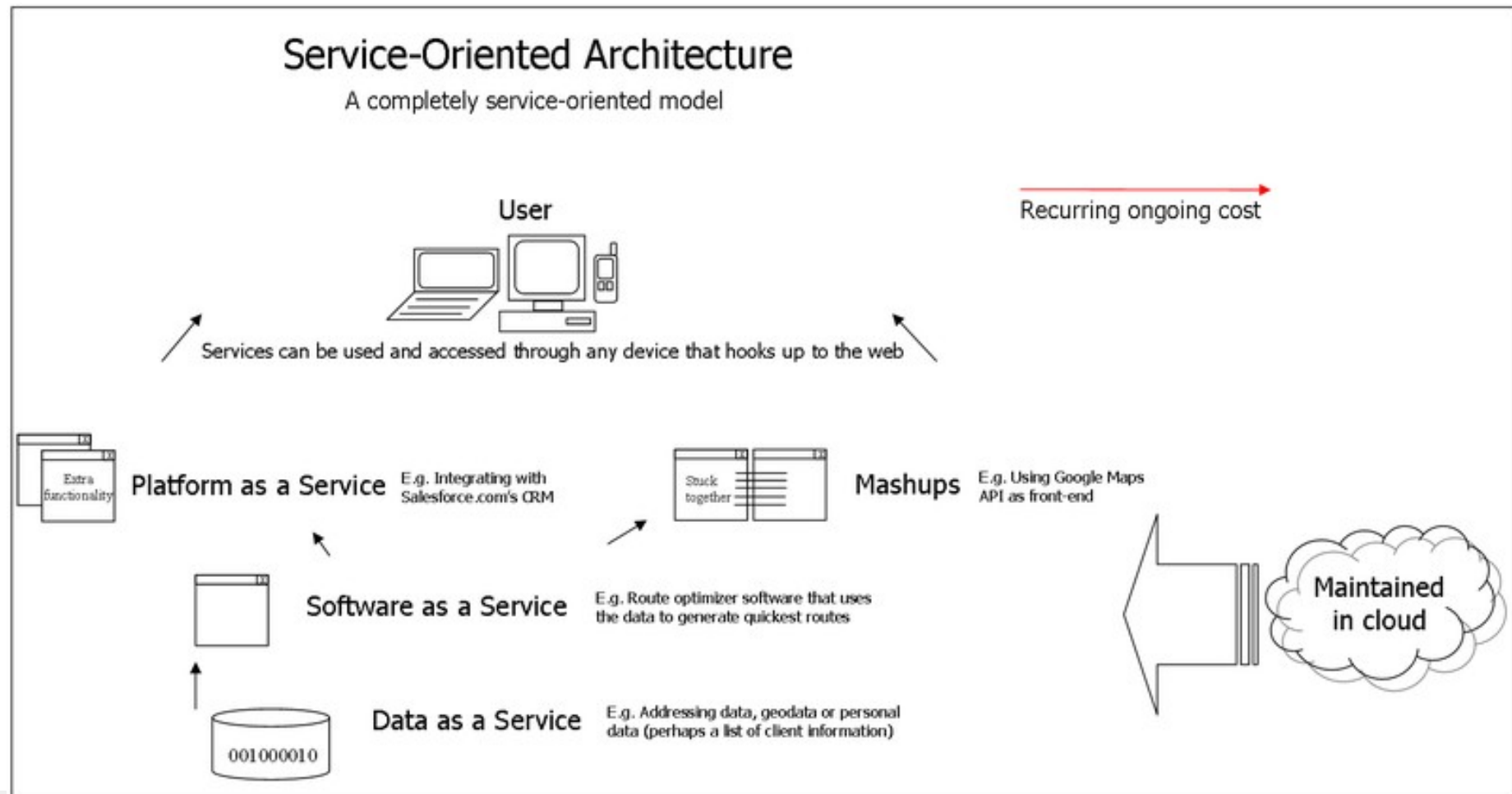
Classical Web Services - SOAP + WSDL

Web Services are:

- components for building distributed applications in SOA architectural style
- communicate using open protocols
- are self-descriptive and self-content
- can be searched and found using UDDI or ebXML registries (and more recent specifications – WSIL & **Semantic Web Services**)



Service Oriented Architecture (SOA). Mashups



Source: http://en.wikipedia.org/wiki/File:SOA_Detailed_Diagram.png,
Author: JamesLWilliams2010, License: Creative Commons Attribution 3.0 Unported

Representational State Transfer (REST) [1]

- REpresentational State Transfer (REST) is an architecture for accessing distributed hypermedia web-services
- The resources are identified by URIs and are accessed and manipulated using an HTTP interface base methods (GET, POST, PUT, DELETE, OPTIONS, HEAD, PATCH)
- Information is exchanged using representations of these resources
- Lightweight alternative to SOAP+WSDL -> HTTP + Any representation format (e.g. JavaScript™ Object Notation – JSON)



Representational State Transfer (REST) [2]

- Identification of resources – URIs
- Representation of resources – e.g. HTML, XML, JSON, etc.
- Manipulation of resources through these representations
- Self-descriptive messages - Internet media type (**MIME type**) provides enough information to describe how to process the message. Responses also explicitly indicate their **cacheability**.
- Hypermedia as the engine of application state (aka **HATEOAS**)
- Application contracts are expressed as **media types** and [semantic] link relations (**rel** attribute - RFC5988, "Web Linking")

[Source: http://en.wikipedia.org/wiki/Representational_state_transfer]



Hypermedia As The Engine Of Application State (HATEOAS) – New Link Header (RFC 5988) Example

Content-Length →1656

Content-Type →application/json

Link →<<http://localhost:8080/polling/resources/polls/629>>; **rel="prev"**;
type="application/json"; title="Previous poll",
<<http://localhost:8080/polling/resources/polls/632>>; **rel="next"**;
type="application/json"; title="Next poll",
<<http://localhost:8080/polling/resources/polls>>; **rel="collection"**;
type="application/json"; title="Polls collection",
<<http://localhost:8080/polling/resources/polls>>; **rel="collection up"**;
type="application/json"; title="Self link",
<<http://localhost:8080/polling/resources/polls/630>>; **rel="self"**



Simple Example: URLs + HTTP Methods

Uniform Resource Locator (URL)	GET	PUT	POST	DELETE
Collection, such as http://api.example.com/comments/	List the URIs and perhaps other details of the collection's members.	Replace the entire collection with another collection.	Create a new entry in the collection. The new entry's URI is assigned automatically and is usually returned by the operation.	Delete the entire collection.
Element, such as http://api.example.com/comments/11	Retrieve a representation of the addressed member of the collection, expressed in an appropriate Internet media type.	Replace the addressed member of the collection, or if it does not exist, create it.	Not generally used. Treat the addressed member as a collection in its own right and create a new entry in it.	Delete the addressed member of the collection.

Source: https://en.wikipedia.org/wiki/Representational_state_transfer

Advantages of REST

- **Scalability of component interactions** – through layering the client server-communication and enabling load-balancing, shared caching, security policy enforcement;
- **Generality of interfaces** – allowing simplicity, reliability, security and improved visibility by intermediaries, easy configuration, robustness, and greater efficiency by fully utilizing the capabilities of HTTP protocol;
- **Independent development and evolution of components**, dynamic evolvability of services, without breaking existing clients.
- **Fault tolerant, Recoverable, Secure, Loosely coupled**



Richardson's Maturity Model of Web Services

According to **Leonard Richardson** [Talk at QCon, 2008 - <http://www.crummy.com/writing/speaking/2008-QCon/act3.html>]:

- **Level 0 – POX:** Single URI (XML-RPC, SOAP)
- **Level 1 – Resources:** Many URIs, Single Verb (URI Tunneling)
- **Level 2 – HTTP Verbs:** Many URIs, Many Verbs (CRUD – e.g Amazon S3)
- **Level 3 – Hypermedia Links Control the Application State = HATEOAS (Hypertext As The Engine Of Application State) == **truely** RESTful Services**



Web Application Description Language (WADL)

- XML-based file format providing machine-readable description of HTTP-based web application resources – typically RESTful web services
- WADL is a W3C Member Submission
 - Multiple resources
 - Inter-connections between resources
 - HTTP methods that can be applied accessing each resource
 - Expected inputs, outputs and their data-type formats
 - XML Schema data-type formats for representing the RESTful resources
- But WADL resource description is **static**, while resources change dynamically – REST anti-pattern

RESTful Patterns and Best Practices

According to **Cesare Pautasso**

[<http://www.jopera.org/files/SOA2009-REST-Patterns.pdf>]:

- Uniform Contract
- Content Negotiation
- Entity Endpoint
- Endpoint Redirection
- Distributed Response Caching
- Entity Linking
- Idempotent Capability



REST Antipatterns and Worst Practices

According to **Jacob Kaplan-Moss**

[<http://jacobian.org/writing/rest-worst-practices/>]:

- Conflating models and resources
- Hardcoded authentication
- Resource-specific output formats
- Hardcoded output formats
- Weak HTTP method support (e.g. tunnel everything through GET/POST)
- Improper use of links
- Couple the REST API to the application





Home

Polls ▾

Search

Submit

Link

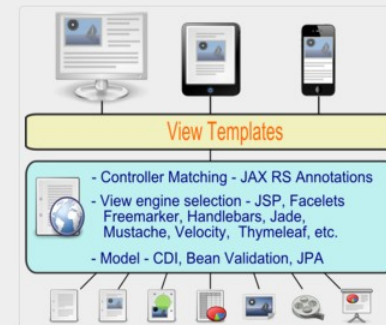
Dropdown ▾

IPT Polling Demo MVC 1.0

IPT Polling Application demonstrates new JavaEE 8 [JSR 371: Model-View-Controller \(MVC 1.0\) Specification](#) and it's reference implementation - project Ozark (<https://ozark.java.net/>). MVC 1.0 is an action-oriented framework building on experience with previous frameworks such as *Struts*, *Struts*, *Spring MVC*, *VRaptor* etc. It is based on *JAX-RS*, *CDI* and *BV JavaEE technologies* and provides a standard, view specification neutral way to build web applications. Among supported view template frameworks are: *JSP*, *Facelets*, *Freemarker*, *Handlebars*, *Jade*, *Mustache*, *Velocity*, *Thymeleaf*, etc.

NOTE: MVC 1.0 API Specification is in early draft stage, and is subject to change based on open community process.

Create Your Poll ...



Recent Polls

BGOUG Poll

Which track do you prefer?

Java MVC Frameworks

Which MVC you choose?

Open Cloud Stacks

Which open cloud stack do you p

JavaScript Libraries

Which is your favorite JS library?



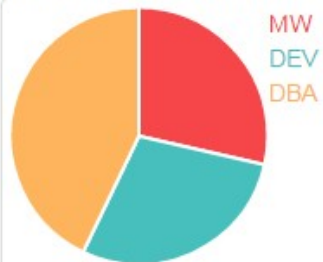
Licensed under the **Creative Commons Attribution-NonCommercial-NoDerivs 3.0 Unported License**

Slide 25

Recent Polls

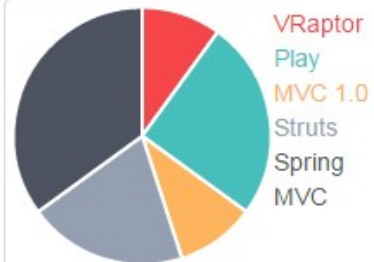
BGOUG Poll

Which track do you prefer?



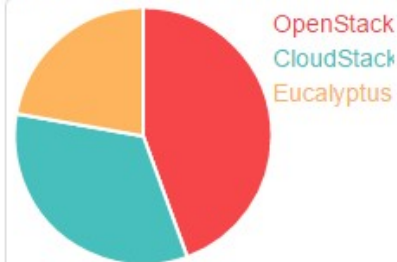
Java MVC Frameworks

Which MVC you choose?



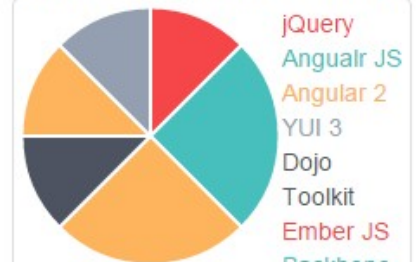
Open Cloud Stacks

Which open cloud stack do you p



JavaScript Libraries

Which is your favorite JS library?



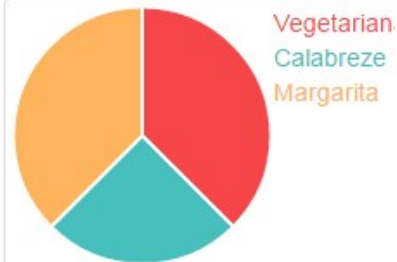
JavaScript Libraries

Which is your favorite JS library?

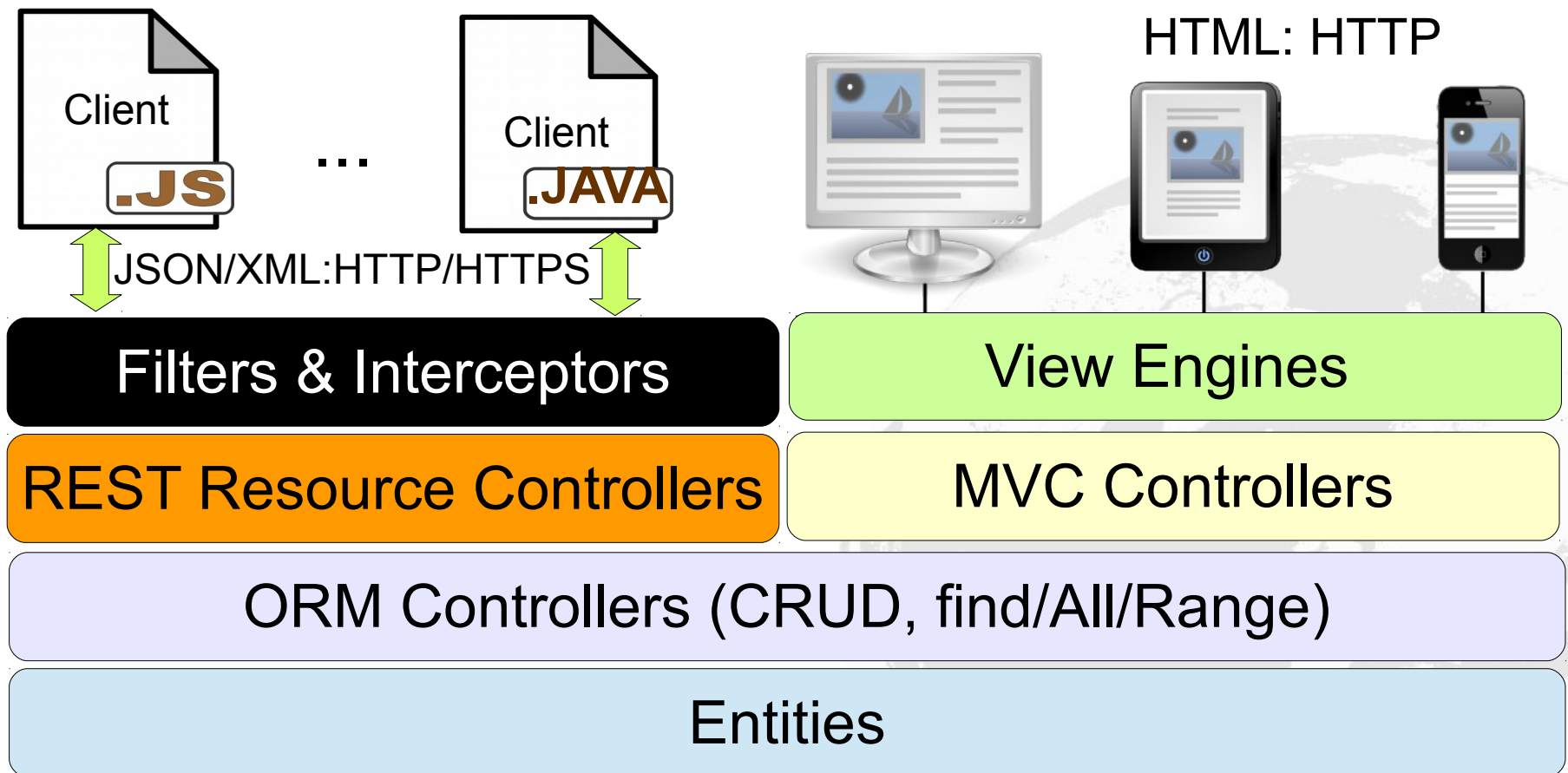


Pizzas

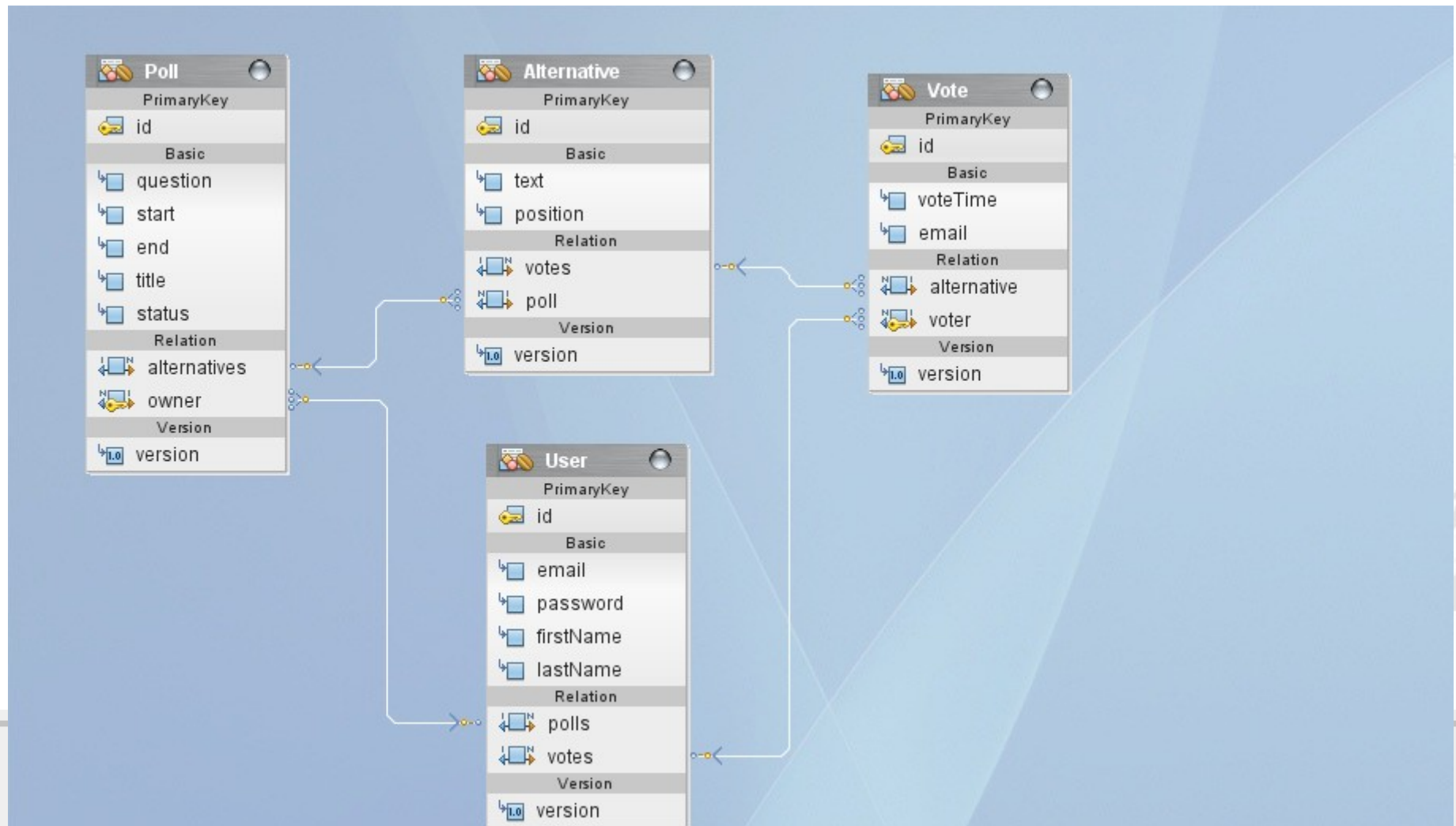
What is your favorite pizza?



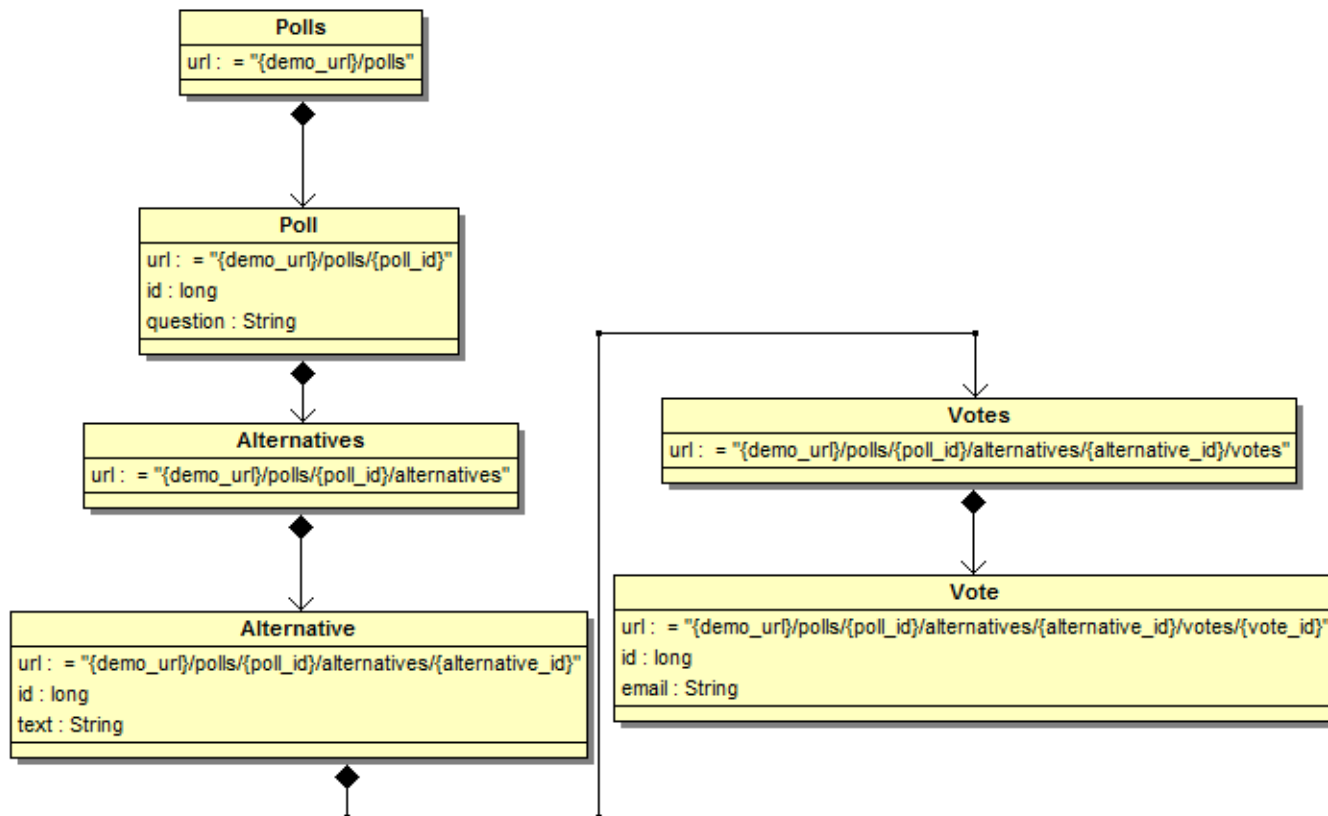
N-Tier Architectures



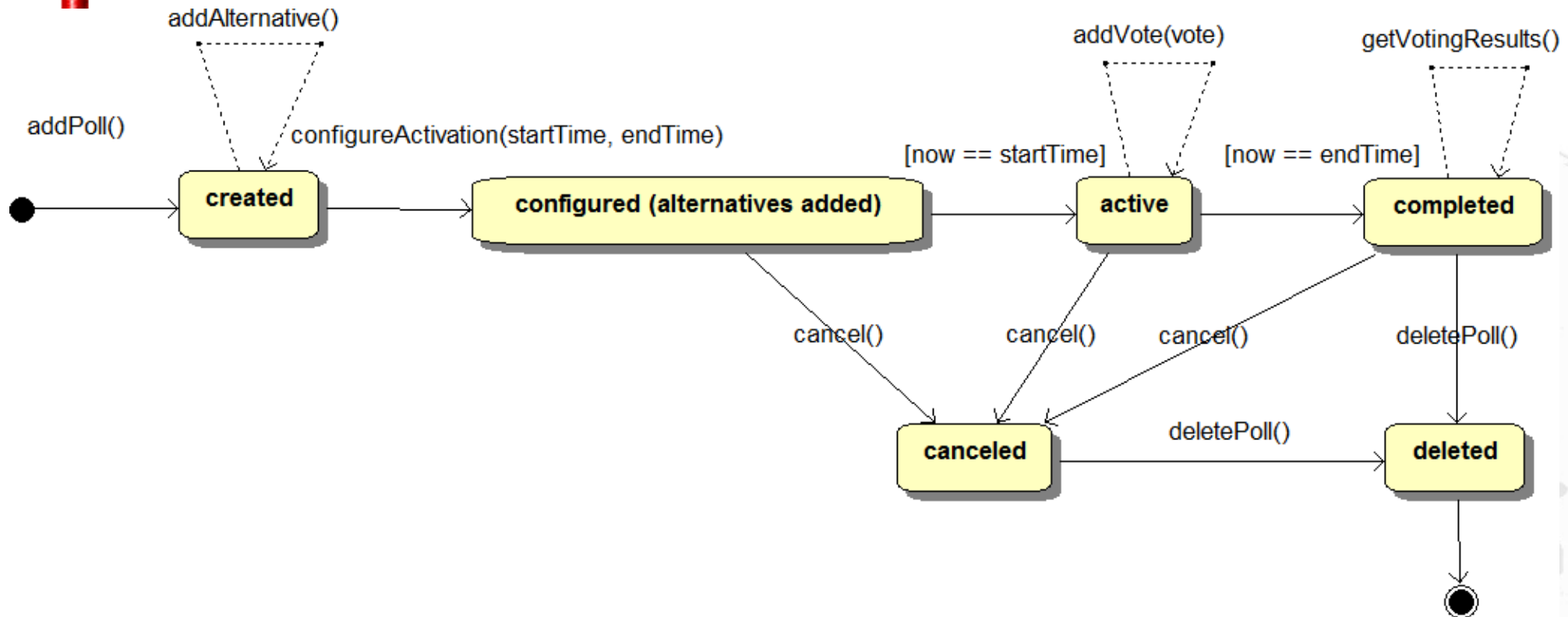
IPT Polling Demo MVC 1.0 Data Architecture



IPT Polling Demo Resources



IPT Polling Demo – Poll Resource States



IPT Polling Demo

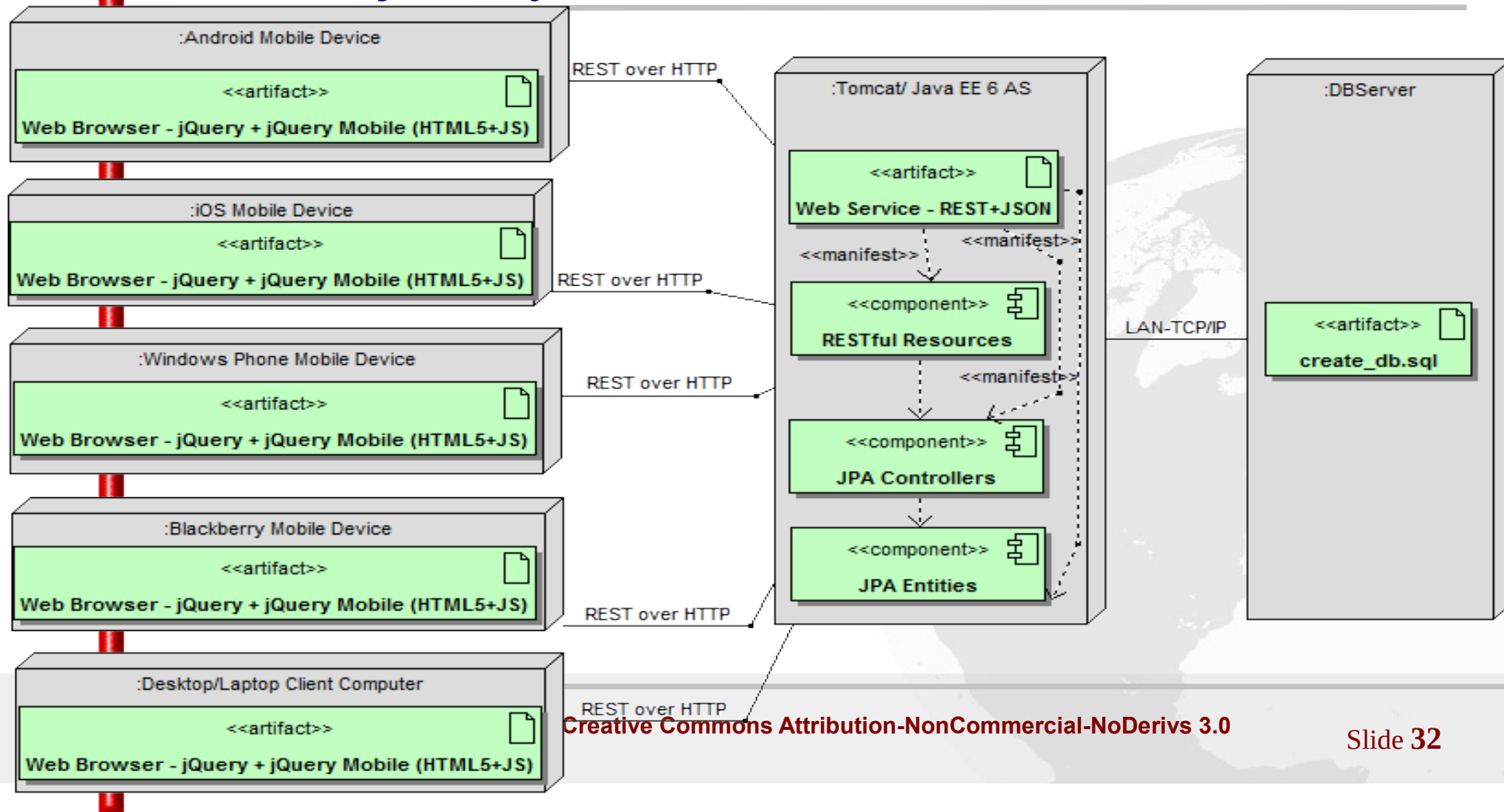
The screenshot shows the Postman application interface. The top bar displays the browser address bar with the URL `chrome-extension://fdmmgilgnpjigdojopjoooidkmcomcm/index.html`. The Postman interface includes a sidebar on the left with a 'History' tab and a list of recent requests. The main area shows a request to `http://localhost:8080/polling/resources/polls/630` using the GET method. The response is a JSON object with the following structure:

```

1 {
2   "id": 630,
3   "status": "CREATED",
4   "title": "new",
5   "question": "one",
6   "start": "2002-05-30T09:00:00+02:00",
7   "end": "2002-05-30T09:00:00+02:00",
8   "_embedded": {
9     "alternative": [
10      {
11        "id": 1,
12        "text": "alt1",
13        "position": 1,
14        "votes": 7,
15        "_links": {
16          "links": [
17            {
18              "href": "http://localhost:8080/polling/resources/polls/630/alternatives/1",
19              "rel": "self",
20              "title": "Self link",
21              "type": "application/json"
22            }
23          ]
24        }
25      },
26      {
27        "id": 6,
28        "text": "altX",
29        "position": 1,
30        "votes": 0,
31        "_links": {
32          "links": [
33            {
34              "href": "http://localhost:8080/polling/resources/polls/630/alternatives/6",
35              "rel": "self",
36              "title": "Self link",
37              "type": "application/json"
38            }
39          ]
40        }
41      }
42    ]
43  }
44 }
  
```

The response status is 200 OK and the time taken is 10659 ms. The response body is displayed in a 'Pretty' format. The bottom of the screen shows a Windows taskbar with various icons and the system clock.

Proposed Architecture: jQuery Client + RESTful Backend



References

- R. Fielding, Architectural Styles and the Design of Networkbased Software Architectures, PhD Thesis, University of California, Irvine, 2000
- Fielding's blog discussing REST – <http://roy.gbiv.com/untangled/2008/rest-apis-must-be-hypertext-driven>
- Representational state transfer (REST) in Wikipedia – http://en.wikipedia.org/wiki/Representational_state_transfer
- Hypermedia as the Engine of Application State (HATEOAS) in Wikipedia – <http://en.wikipedia.org/wiki/HATEOAS>
- JavaScript Object Notation (JSON) – <http://www.json.org/>
- JAX-RS 2.0 (JSR 339) Reference Implementation for building RESTful Web services – <http://jersey.java.net/>



Thanks for Your Attention!

Questions?

